

Программирование на С

Лекция 2

Разбираем первую программу

Наша первая программа

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

Первая строка

```
#include <stdio.h>
```

- сообщает компилятору, что нам понадобится библиотека для работы с экраном;
- библиотека уже написана другими программистами;
- мы просто говорим: «разреши ей пользоваться».

Пока считайте эту строку магическим заклинанием.

Вторая строка

```
int main(void)
```

Здесь начинается *функция* `main` .

Выполнение программы всегда начинается с `main`.

Фигурные скобки

{ и }

Они отмечают начало и конец программы, которую выполняет *функция* `main` .

Главная команда

```
printf("Hello, world!\n");
```

Команда `printf` выводит текст на экран.

Что означает `\n`

```
printf("Hello, world!\n");
```

`\n` означает: перейти на новую строку.

Поэтому следующий вывод начнётся уже с новой строки.

Последняя строка

```
return 0;
```

После выполнения программы управление возвращается операционной системе.

Число `0` означает:

программа завершилась успешно.

Пока это тоже можно считать небольшим магическим ритуалом.

Комментарии

Комментарии — это пояснения для человека.

Компилятор их игнорирует.

```
// Однострочный комментарий
```

```
/*  
    Многострочный  
    комментарий  
*/
```

Комментарии помогают понять программу,
но не влияют на её работу.

Комментарии

Комментарии нужны

- **Заглавный комментарий.** Размещайте заглавный комментарий, который описывает назначение файла, вверху каждого файла.
- **Заголовок функции.** Разместите заголовочный комментарий на каждой функции вашего файла. Заголовок должен описывать поведение и/или цель функции.
- **Параметры / возврат.** Если ваша функция принимает параметры, то кратко опишите их цель и смысл. Если ваша функция возвращает значение — кратко опишите, что она возвращает.
- **Комментарии на одной строке.** Если внутри функции имеется секция кода, которая длинна, сложна или непонятна, то кратко опишите её назначение.

Комментарии

Но...

Сложный код, написанный с использованием хитрых ходов, следует не комментировать, а переписывать!

Следует делать как можно меньше комментариев, делая код самодокументируемым путём выбора правильных имён и создания ясной логической структуры.